Minimizing fStress and rStress by Majorizing Gauss-Newton

Jan de Leeuw

March 7, 2026

TBD

Table of contents

1 Loss Functions **3**

2 Algorithm **4**
2.1 Normalized Cone Regression 4
2.2 Majorization 5

3 Software **8**
3.1 Data 8
3.2 Initial Estimate 8
3.3 Main 9

4 Examples **10**

References **33**

Note: This is a working manuscript which will be expanded/updated frequently. All suggestions for improvement are welcome. All Rmd, tex, html, pdf, R, and C files are in the public domain. Attribution will be appreciated, but is not required. The files can be found at https://github.com/deleeuw/rStress

1 Loss Functions

The Multidimensional Scaling (MDS) loss function fStress is defined as

$$\sigma_f(x) := \frac{1}{2}\sum_{k=1}^{K} w_k(f(\delta_k) - f(d_k(x)))^2, \tag{1}$$

with f increasing and differentiable in the open interval $(0, +\infty)$. In (1) the w_k are positive *weights*, the δ_k are known *dissimilarities*. The vector x has the coordinates of n points in $\mathbb{R}^p$, and the $d_k(x)$ are Euclidean *distances* between pairs of these points. For each k there is a pair of indices (i, j) with $1 \leq i < j \leq n$ and a matrix A_k which is the direct sum of p copies of $(e_i - e_j)(e_i - e_j)'$, where the e_i are unit vectors (columns of the identity matrix of order n). Thus $d_k(x) = \sqrt{x'A_kx}$. Metric least squares MDS minimizes fStress over x.

fStress was first introduced and studied in Groenen, De Leeuw, and Mathar (1995). No explicit algorithm to minimize loss (**??**)eq-fdef) was given, but the paper has formulas for the first and second derivatives (see also De Leeuw (2017)), which can of course be used in general purpose optimization routines.

An important special case of fStress is rStress (also known as powerStress), which is

$$\sigma_r(x) := \frac{1}{2}\sum_{k=1}^{K} w_k(\delta_k^r - d_k^r(x))^2 \tag{2}$$

Special cases of rStress are Kruskal's stress (Kruskal (1964a), Kruskal (1964b)) with $r = 1$, sstress by Takane, Young, and De Leeuw (1977) with $r = 2$, and the limiting case logarithmic stress with $r \rightarrow 0$ by Ramsay (1977). There have been various attempts to extend the majorization (or MM) method for MDS (De Leeuw (1977)) to rStress. References and links to various unpublished reports are in De Leeuw (2017). The recent smacofx package (Rusch et al. (In Press)) has R code for the rstressMin() function that implements the majorization method in De Leeuw, Groenen, and Mair (2016).

Minimizing of either fStress or rStress over x is a metric MDS problem. The δ_k, and consequently the $f(\delta_k)$, are K known numbers. In non-metric MDS the loss function is minimized over both x and δ, where δ is constrained to be in a subset Δ of $\mathbb{R}^K$. For the ordinal version of non-metric MDS, for example, we require $\delta_1 \leq \cdots \leq \delta_K$. Since f is increasing, we can write fStress simply as

$$\sigma_f(x, \delta) := \frac{1}{2}\sum_{k=1}^{K} w_k(\delta_k - f(d_k(x)))^2, \tag{3}$$

where δ is no longer a vector of known dissimilarities, but any vector monotone with the dissimilarities. These transformed or scaled dissimilarities are often called *disparities*. Non-metric rStress is simply (3) with $f(d_k(x)) = d_k^r(x)$.

In order to exclude the trivial solution with $x = 0$ and $\delta = 0$ in addition we impose the normalization constraint

$$\eta^2(\delta) := \frac{1}{2}\sum_{k=1}^{K} w_k \delta_k^2 = 1. \tag{4}$$

This formulation of non-metric MDS can be generalized to $\delta \in \Delta$, where Δ is a convex cone in $\mathbb{R}^K$. This means we require the disparities to be in the intersection of the cone Δ and the sphere Σ defined by (4).

It is convenient to think of metric MDS as the special case in which the set Δ is the one-element set containing only the normalized dissimilarities δ. Alternatively, as in smacof, we can choose for Δ the ray of all vectors that are non-negative multiples of the dissimilarities. In other words our metric MDS treats the dissimilarities as measured on a ratio scale.

2 Algorithm

The technique proposed in this paper to minimize non-metric fStress or rStress is in the Alternating Least Squares (ALS) family. We start with an initial estimate $x^{(0)}$. We then minimize σ_f over δ in $\Delta \cap \Sigma$ for the current $d(x^{(0)})$. The minimizer $\delta^{(0)}$ is then used to minimize fStress over x for the current disparities, yielding $x^{(1)}$. These two steps are alternated until x and δ do not change any more. Starting in iteration $\nu = 0$ we compute

$$\delta^{(\nu)} = \underset{\delta \in \Delta \cap \Sigma}{\operatorname{argmin}}\, \sigma_f(x^{(\nu)}, \delta), \tag{5a}$$

$$x^{(\nu+1)} = \underset{x}{\operatorname{argmin}}\, \sigma_f(x, \delta^{(\nu)}). \tag{5b}$$

We then increase ν by one and go into the next iteration. And so on, until convergence.

2.1 Normalized Cone Regression

In metric MDS there is no need for the first step (5a), because $\delta^{(\nu)}$ is equal to the normalized dissimilarities δ for all ν. In the non-metric case step (5a) is needed, but compared to (5b) it is comparatively easy. We have to compute the least squares projection of $f(d(x))$ on the cone Δ and then normalize this projection to give it length one. If Δ is the cone of monotone vectors, as in ordinal MDS, the cone projection is *monotone regression*. The fact that projection on the intersection of Δ and Σ is the same thing as normalizing the projection on Δ is due to De Leeuw (1975) and more generally (and more rigorously) to Bauschke, Bui, and Wang (2018). Since this step is the same as in standard MDS algorithms such as smacof (De Leeuw and Mair (2009), Mair, Groenen, and De Leeuw (2022)) we do not go into details here, and just refer to the literature.

2.2 Majorization

The second step (5b), minimizing over x for fixed current δ, is more complicated. There is no analytic solution, unlike the first step, and minimization requires an iterative process of its own. Thus, except in some very special cases, the second step requires an infinite number of "inner" iterations. Since implementing an infinite number of iterations is impossible we have to truncate the inner iteration sequence at some point. In our software we deviate from the strict ALS framework by not minimizing fStress over x for fixed δ, but by taking a single "inner" iteration and merely decrease fStress. If this is done judiciously we still obtain a convergent sequence of updates. This is similar to the strategy in other MDS algorithms such as smacof (De Leeuw (1977)) and alscal (Takane, Young, and De Leeuw (1977)).

In smacof the inner iteration step is a majorization step, by now more commonly known as an MM step. Briefly, we find a function $\kappa_f(x;y)$ such that

- $\kappa_f(x,y) \leq \sigma_f(x)$ for all x and y in Ξ, and
- $\kappa_f(x;y) = \sigma_f(x)$ if and only if $x = y$.

An inner iteration is of the form

$$x^{(\mu+1)} = \underset{x\in\Xi}{\text{argmin}}\, \kappa(x, x^{(\mu)}). \tag{6}$$

If $x^{(\mu+1)} = x^{(\mu)}$ we declare convergence and stop. From (6),

$$\sigma_f(x^{(\mu+1)}) \leq \kappa_f(x^{(\mu+1)}, x^{(\mu)}) < \kappa_f(x^{(\mu)}, x^{(\mu)}) = \sigma_f(x^{(\mu)}). \tag{7}$$

Thus an inner majorization step upgrading x for given δ decreases fStress (unless we stop). Remember that in ALS the inner iterative process (indexed by μ) goes on within step 2 of the "outer" update (indexed by ν).

In De Leeuw, Groenen, and Mair (2016) a majorization method was proposed to minimize rStress. It majorizes d^r as a function of x, and for this it uses the specific properties of the power function. It consequently cannot be used for fStress. The technique is incorporated in the smacofx R package, and numerical experience indicates it works, but convergence can be painfully slow. Because of the generality of the function f (any differentiable increasing function) it seems impossible to develop an majorization method for fStress along the same lines as the one for rStress. Consequently we go a somewhat different route in this paper. We do not only deviate from the strict ALS procedure by only partially minimizing loss over x for fixed d, we also deviate from the strict majorization approach by majorizing an approximation of fStress.

In deriving the approximation and majorization we surpress the dependency of fStress on δ, because in the second ALS substep δ is just a vector of constants. We first approximate $f(d(x))$ near $d(y)$ with

$$f(d_k(x)) \approx f(d_k(y)) + \mathcal{D}f(d_k(y))(d_k(x) - d_k(y)). \tag{8}$$

Define

$$\omega_f(x;y) := \sum_{k=1}^{K} w_k(f(\delta_k) - f(d_k(y)) - \mathcal{D}f(d_k(y))(d_k(x) - d_k(y)))^2, \tag{9}$$

Note that in (8) the derivative is with respect to d, not with respect to x or y. This is a major difference with the majorizations in De Leeuw, Groenen, and Mair (2016).

Note that $\omega_f(x;x) = \sigma_f(x)$ for all x and if f is the identity then $\omega_f(x;y) = \sigma_f(x)$ for all x and y. If f is linear with intercept β and slope α then

$$\omega_f(x,y) = a^2 \sum_{k=1}^{K} w_k(\delta_k - d_k(x))^2 = a^2\sigma_r(x), \tag{10}$$

with $r = 1$.

We now give an alternative and more convenient expression for $\omega_f(x,y)$. Define

$$\tilde{w}_k(y) := w_k\{\mathcal{D}f(d_k(y))\}^2, \tag{11a}$$

$$\tilde{\delta}_k(y) := \frac{f(\delta_k) - f(d_k(y))}{\mathcal{D}f(d_k(y))} + d_k(y). \tag{11b}$$

Then

$$\omega_f(x;y) = \sum_{k=1}^{K} \tilde{w}_k(y)(\tilde{\delta}_k(y) - d_k(x))^2. \tag{12}$$

We see from (11a) that $\tilde{w}_k(y)$ is always non-negative and from (11b) that $\tilde{\delta}_k(y)$ can be negative if $d_k(y) > \delta_k$. For convex f, however, we have

$$\tilde{\delta}_k(y) = \frac{f(\delta_k) - f(d_k(y))}{\mathcal{D}f(d_k(y))} + d_k(y) \geq d_k(x) \geq 0. \tag{13}$$

For rStress equations (11a) and (11b) become

$$\tilde{w}_k(y) := r^2 w_k d_k^{2(r-1)}(y), \tag{14a}$$

$$\tilde{\delta}_k(y) := \frac{\delta_k^r + (r-1)d_k^r(y)}{r d_k^{r-1}(y)}. \tag{14b}$$

By convexity $\tilde{\delta}_k(y)$ is non-negative if $r \geq 1$. For $0 < r < 1$ we have $\tilde{\delta}_k(y) > 0$ if and only if $d_k(y) < (1-r)^r \delta_k$. If $\delta_k > d_k(y)$ then $\tilde{\delta}_k(y) > d_k(y) \geq 0$.

By writing $\omega_f(x, y)$ as in (12) we are on familiar majorization terrain. If $\delta_k(y) > 0$ we use Cauchy-Schwartz, as in standard smacof,

$$d_k(x) \geq \frac{1}{d_k(y)} \text{tr}\, x' A_k y, \tag{15a}$$

and if $\delta_k(y) < 0$ we use the arithmetic mean/geometric mean (AM/GM) inequality in the form

$$d_k(x) \leq \frac{1}{2} \frac{1}{d_k(y)} \{y' A_k y + x' A_k x\}, \tag{15b}$$

Use of the AM/GM inequality for majorizing terms with negative dissimilarities was pioneered by Heiser (1991).

Define the matrices

$$V(y) := \sum_{k=1}^{K} \tilde{w}_k(y) A_k, \tag{16a}$$

$$B(y) := \sum_{\tilde{\delta}_k(y)>0} \tilde{w}_k \frac{\tilde{\delta}_k(y)}{d_k(y)} A_k, \tag{16b}$$

$$H(y) := \sum_{\tilde{\delta}_k(y)<0} \tilde{w}_k \frac{\tilde{\delta}_k(y)}{d_k(y)} A_k. \tag{16c}$$

Note that all three matrices (more preecisely matrix-valued functions) are positive semi-definite. We can use these matrices for a majorization of $\omega_f(x; y)$ at y is

$$\omega_f(x; y) = C + \frac{1}{2} x'(V(y) + H(y))x - x' B(y) x \leq \kappa_f(x; y), \tag{17}$$

with

$$\kappa_f(x; y) := C + \frac{1}{2} x'(V(y) + H(y))x - x' B(y) y, \tag{18}$$

where C is a constant that depends on y but not on x.

We now update using

$$x^{(\mu+1)} = \underset{x}{\text{argmin}}\, \kappa_f(x; x^{(\mu)}) = (V(x^{(\mu)}) + H(x^{(\mu)}))^+ B(x^{(\mu)}) x^{(\mu)},$$

which results in

$$\sigma_f(x^{(\mu+1)}) \approx \omega_f(x^{(\mu+1)}; x^{(\mu)}) \leq \kappa_f(x^{(\mu+1)}; x^{(\mu)}) < \kappa_f(x^{(\mu)}; x^{(\mu)}) = \sigma_f(x^{(\mu)}).$$

Unlike in smacof there is no guarantee that σ_f decreases from one iteration to the other. Monotone convergence of loss function values will only happen if the approximation is good, i.e. if $d(x^{(\mu+1)})$ is close to $d_k(x^{(\mu)})$. This requires closer monitoring of convergence than in smacof. Also note that if $\tilde{\delta}_k(y) < 0$ for all k then $B(y) = 0$, which means that the update from y is equal to zero. Again, this eventuality must be monitored.

3 Software

There are two different major routines in the repository. The R function fStress.R does metric MDS minimizing fStress with an arbitrary f. The C function smacofSSRStress.c does either metric or ordinal MDS minimizing rStress for an arabitrary r. The C code is compiled into a shared library smacofSSRStress.so, which is loaded by the R front-end smacofSSRStress.R that contains smacofSSRStress(). The function fStress() is meant for exploration and research, while smacofSSRStress() is more suitable for production. fStress() is more general, in that it allows for more transformations, but smacofSSRStress() is more general in that it allows for ordinal MDS.

3.1 Data

For fStress.R the input is a matrix delta of dissimilarities. For smacofSSRStress.R the data and weights are in the MDS data format defined in De Leeuw (2025). This is a list with two scalars nobj and ndat, the number of objects and the number of data points (object pairs), and five vectors iind, jind, delta, blocks, weights of length ndat. Vectors iind and jind have integers between 1 and n with iind > jind that code for which pairs of objects we have dissimilarity information. The dissimilarities are in delta, the weights in weights. Dissimilarities are sorted in increasing order and the blocks vectors codes the tie-blocks in the dissimilarities.

3.2 Initial Estimate

There are two possibilities. If xinit is not NULL then it is used as the initial estimate of the configuration. This is useful, for example, if we start the iterations for, say, $r = \frac{1}{2}$ with the solition for $r = 1$. If xinit is NULL then the function smacofSSRStressInit() computes an initial estimate using classical MDS (also known as Torgerson-Gower MDS) on the dissimilarities.

First we make sure the weights add up to one. If there are missing dissimilarities they are first imputed by setting them equal to the weighted mean of the non-missing dissimilarities. Then

we double center the squared dissimilarities and compute the classical scaling solution using the eigs_sym() function from the RSpectra package (Qiu and Mei (2024)).

Now we scale the solution, which is done with the R function smacofSSRStressScale(). We first normalize the powered dissimilarities such that their weighted sum of squares is equal to one. We then choose λ to minimize

$$\sigma(\lambda) := \sum_k w_k(\delta_k^r - d_k^r(\lambda x))^2 = \sum_k w_k(\delta_k^r - \lambda^r d_k^r(x))^2,$$

where x is the Torgerson-Gower solution. The minimum is attained for

$$\hat{\lambda} := \left[\frac{\sum_k w_k \delta_k^r d_k^r(x)}{\sum_k w_k d_k^{2r}(x)}\right]^{1/r}.$$

Our initial estimate is $\hat{\lambda}x$ with distances $\hat{\lambda}d_k(x)$. This scaling is done also if xinit is not NULL.

3.3 Main

The arguments of fStress() are

```
function (delta, xini, wght = 1 - diag(nrow(xini)), f = fPower,
    g = gPower, r = 1, itmax = 10000, eps = 1e-06, verbose = TRUE)
NULL
```

Thus fStress() needs an initial configuration. Parameters f and g are functions that compute the transformation f and its derivative. The file fStress.R includes flog, glog, fexp, gexp, and fPower, gPower. Parameter r is included in case f and g need additional arguments (as in the case of fPower and gPower). Since all code is in R, it is easy to add additional transformation functions if needed. The output of fStress() is

```
list(
    delta = delta,
    dhat = delf,
    weightmat = wght,
    conf = xnew,
    confdist = dnew,
    stress = snew,
    niter = itel
  )
```

The arguments of smacofSSRStress() are

```
function (theData, ndim = 2, xinit = NULL, ties = 1, itmax = 10000,
    eps = 1e-06, rpow = 1, digits = 8, width = 10, verbose = TRUE,
    weighted = FALSE, ordinal = FALSE)
NULL
```

The C code follows the conventions of the smacof versions in De Leeuw (2025). There is only a single parameter $rpow$ for the transformation. The output of smacofSSRStress() is

```r
list(
    delta = theData$delta,
    dhat = h$dhat,
    confdist = h$edis,
    conf = matrix(h$xnew, nobj, ndim),
    weightmat = h$wght,
    stress = h$snew,
    ndim = ndim,
    init = xinit,
    niter = h$itel,
    nobj = nobj,
    iind = h$iind,
    jind = h$jind,
    weighted = weighted,
    ordinal = ordinal,
    ties = h$ties
  )
```

The output list can be fed immediately into the smacof plot routines for the configuration plot, the residual plot, the Shepard plot, and the Dist-Dhat plot.

4 Examples

We analyse the colour data from Ekman (1954) with different functions f, first using fStress(), then using smacofSSRStress().

Table 1: fStress() results Ekman Data

	niter	fStress
logarithm	555	0.0872002
power .1	805	0.0002733
power .5	219	0.0059543
power 1	35	0.0172132
power 2	45	0.0328807
power 5	775	0.0203153
exponent	34	0.0085331

Table 2: smacofSSRStress() results Ekman Data, metric MDS

	niter	rStress
power .1	856	0.0002733
power .5	229	0.0059543
power 1	35	0.0172132
power 2	46	0.0328807
power 5	841	0.0203153

We see that for the power transforms the R and C functions arrive at the same solution. The number of iterations is not excatly the same, because of differences in the initial configuration. With smacofSSRStress() we can also do ordinal MDS.

Table 3: smacofSSRStress() results Ekman Data, ordinal MDS

	niter	rStress
power .1	876	0.0000066
power .5	251	0.0001651
power 1	134	0.0005337
power 2	209	0.0009015
power 5	2443	0.0011002

The package microbenchmark (Mersmann (2024)) can be used to compare the R and C versions. For $r = 1$ the C version is about four times faster, for $r = .1$ and $r = 5$ it is about 10 times faster.

```
Warning in microbenchmark(fStress(ekmanMatrix, xini = xinit, f = fPower, : less
accurate nanosecond times to avoid potential integer overflows


Unit: microseconds

   fStress(ekmanMatrix, xini = xinit, f = fPower, g = gPower, r = 1,      verbose = FA
                                    smacofSSRStress(ekmanData, rpow = 1, verbose = FA
 fStress(ekmanMatrix, xini = xinit, f = fPower, g = gPower, r = 0.1,      verbose = FA
                                  smacofSSRStress(ekmanData, rpow = 0.1, verbose = FA
   fStress(ekmanMatrix, xini = xinit, f = fPower, g = gPower, r = 5,      verbose = FA
                                    smacofSSRStress(ekmanData, rpow = 5, verbose = FA
       min        lq       mean     median        uq       max neval cld
  1368.006  1470.280  1782.5049  1522.4735  1563.925  5717.450   100  a
   381.464   395.363   441.3892   412.5625   486.752   607.702   100  a
 33755.013 37817.908 43840.4107 38643.0125 39699.993 83263.620   100   b
  3379.343  3630.325  3670.5963  3684.0140  3720.771  3818.494   100  a
 32748.258 36561.217 45443.7825 37440.7900 39205.676 82820.287   100   b
  3351.299  3514.889  3550.7976  3549.0420  3588.300  3670.361   100  a
```

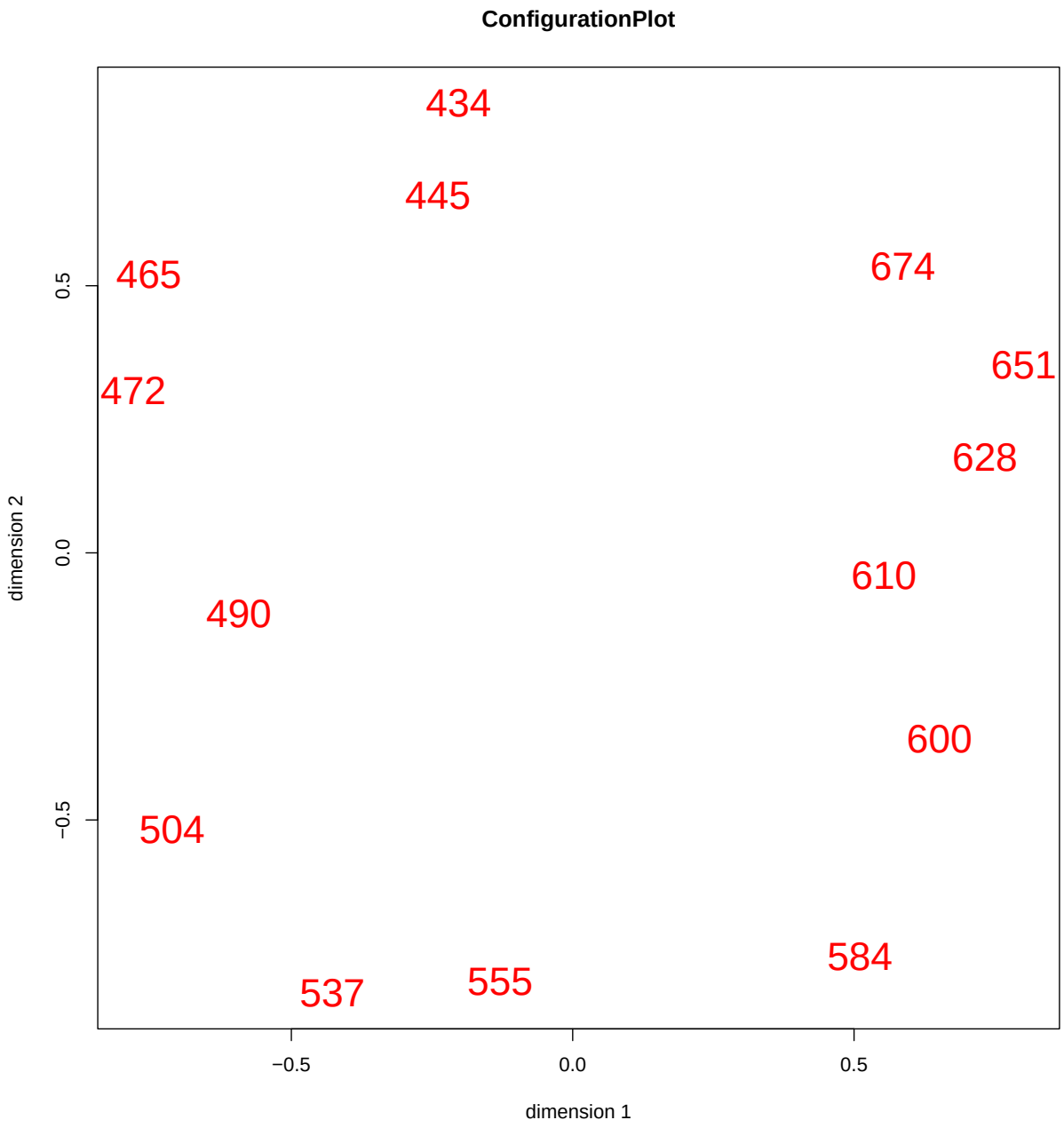

Figure 1: Configuration Plot, Metric, r = .1

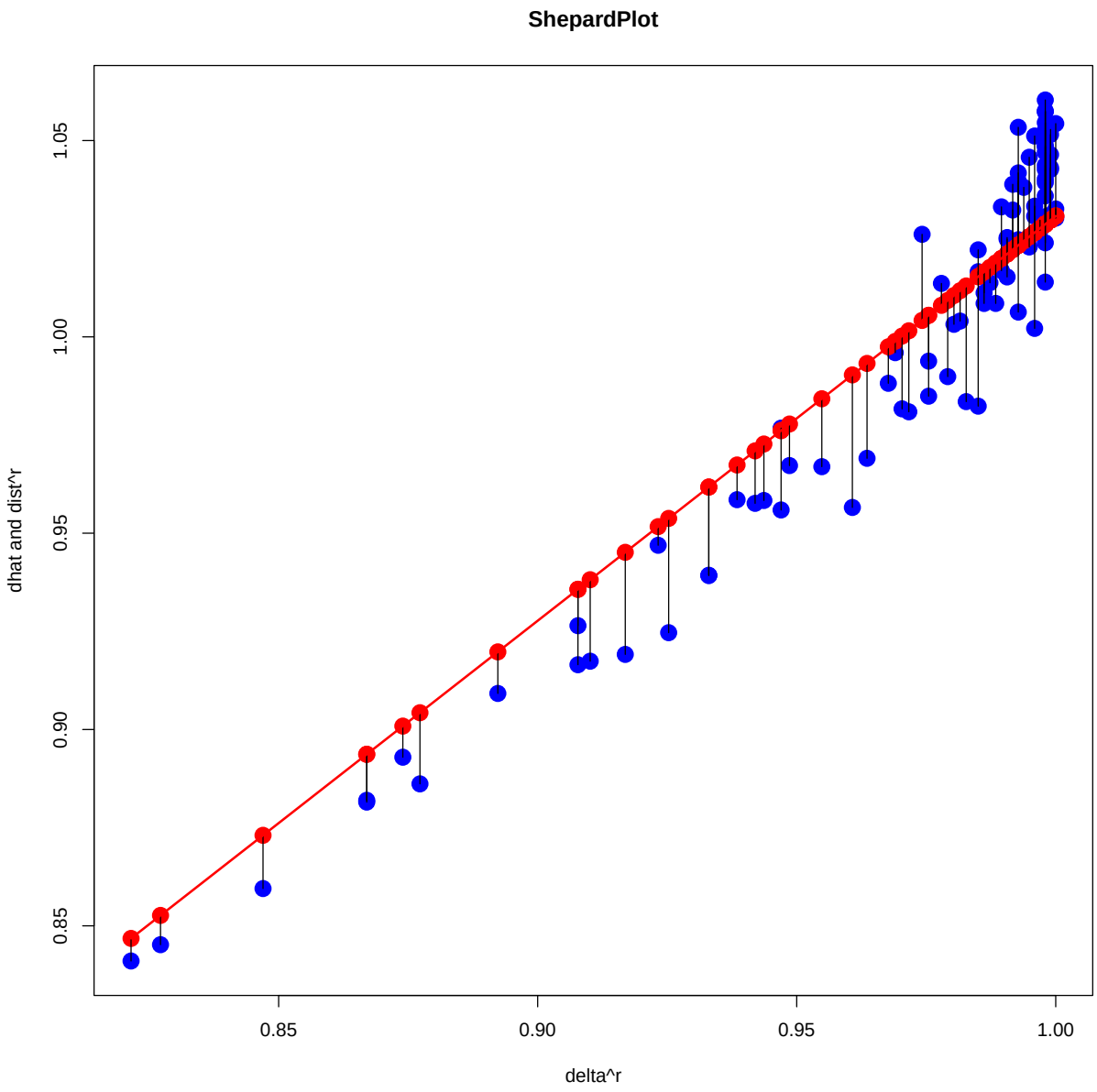

Figure 2: Shepard Plot, Metric, r = .1

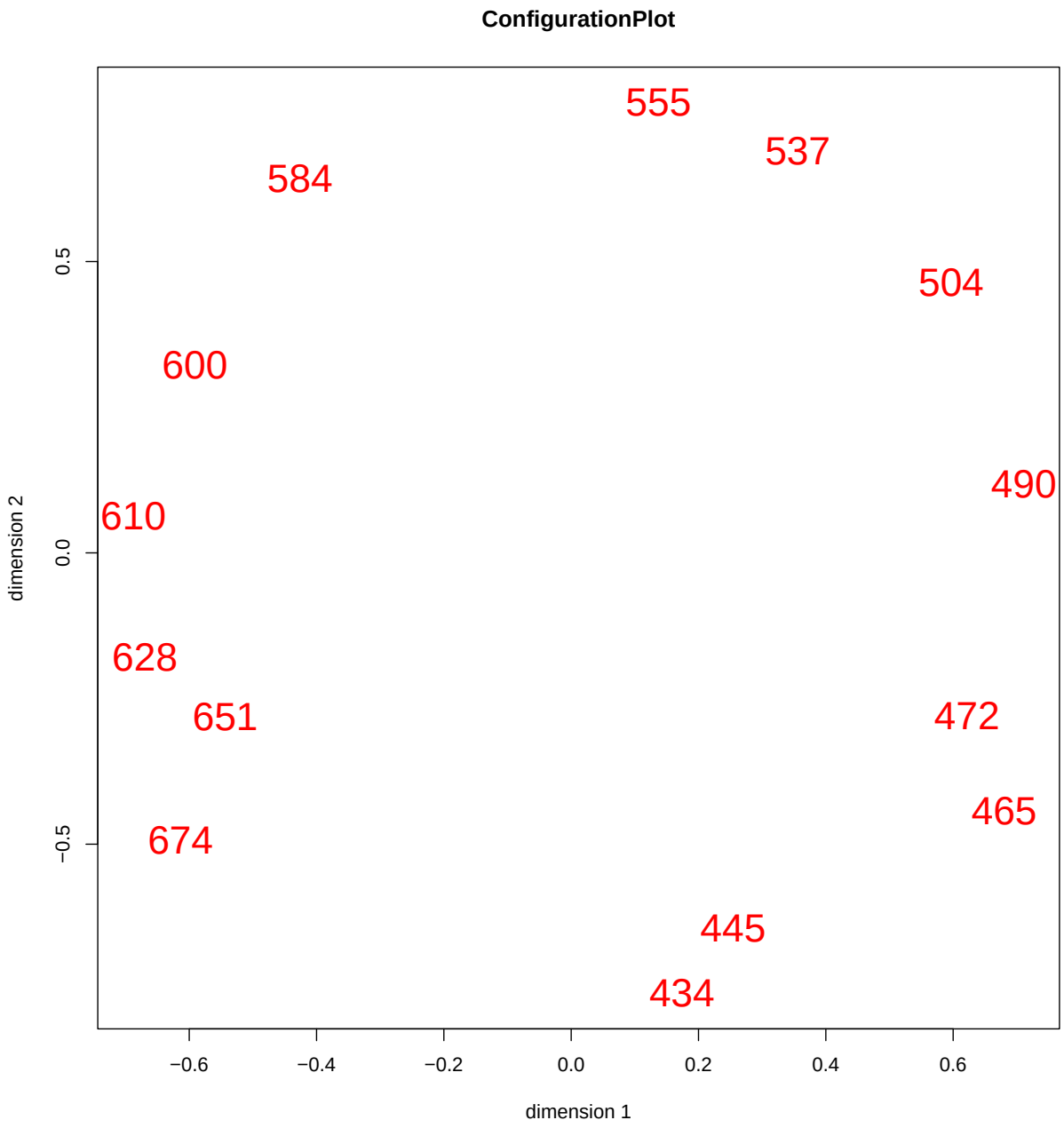

Figure 3: Configuration Plot, Metric, r = .5

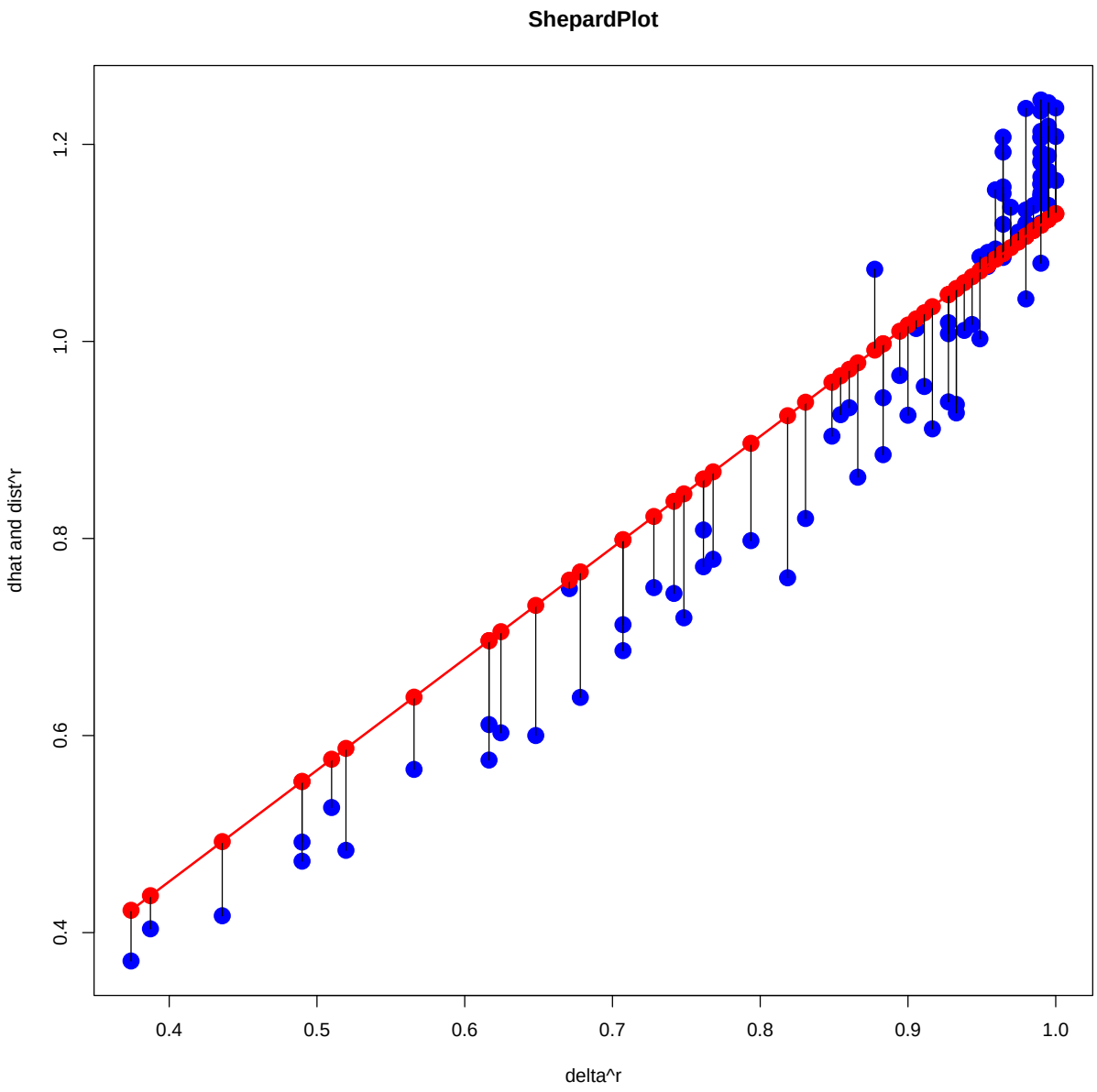

Figure 4: Shepard Plot, Metric, r = .5

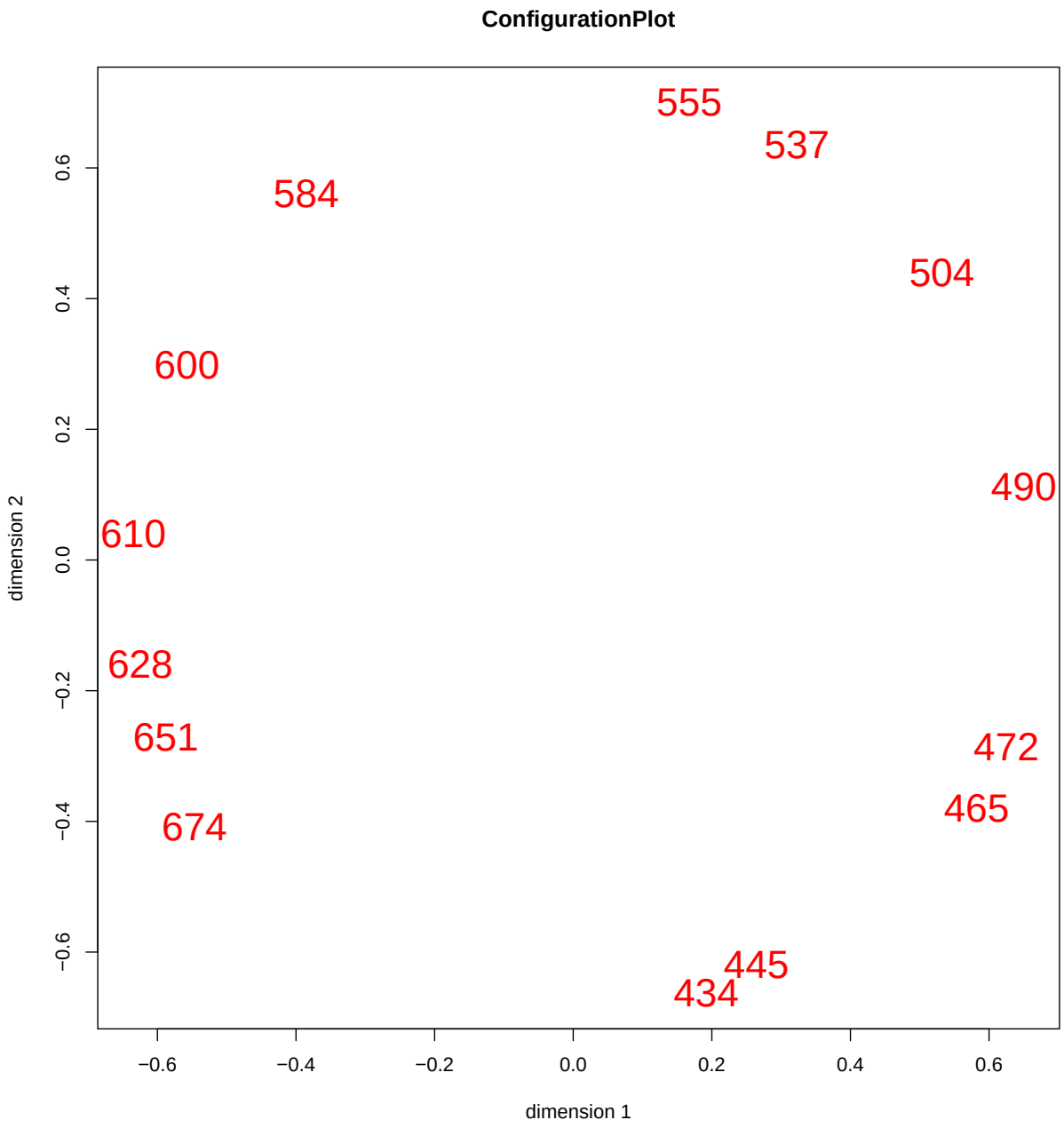

Figure 5: Configuration Plot, Metric, r = 1

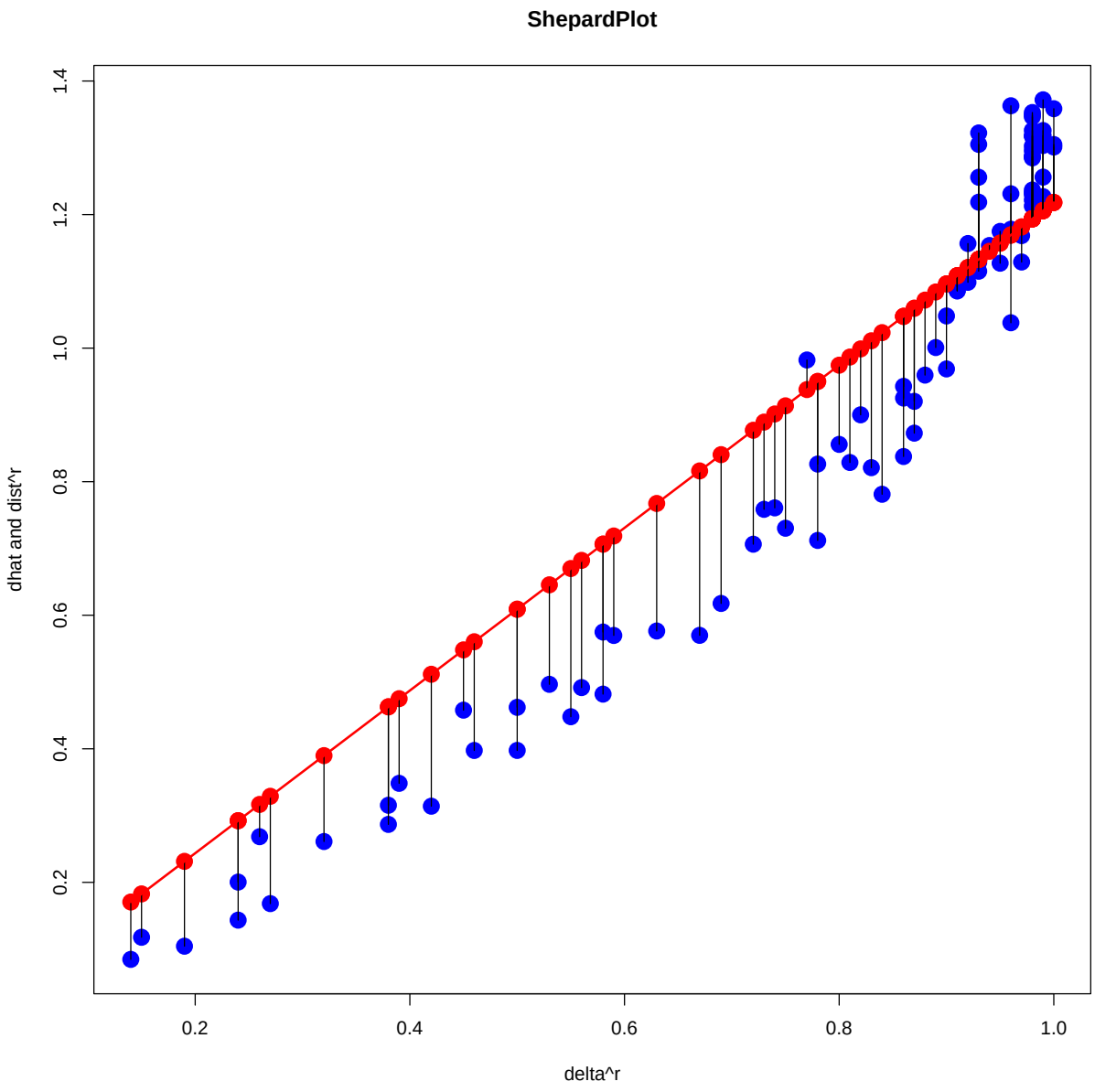

Figure 6: Shepard Plot, Metric, r = 1

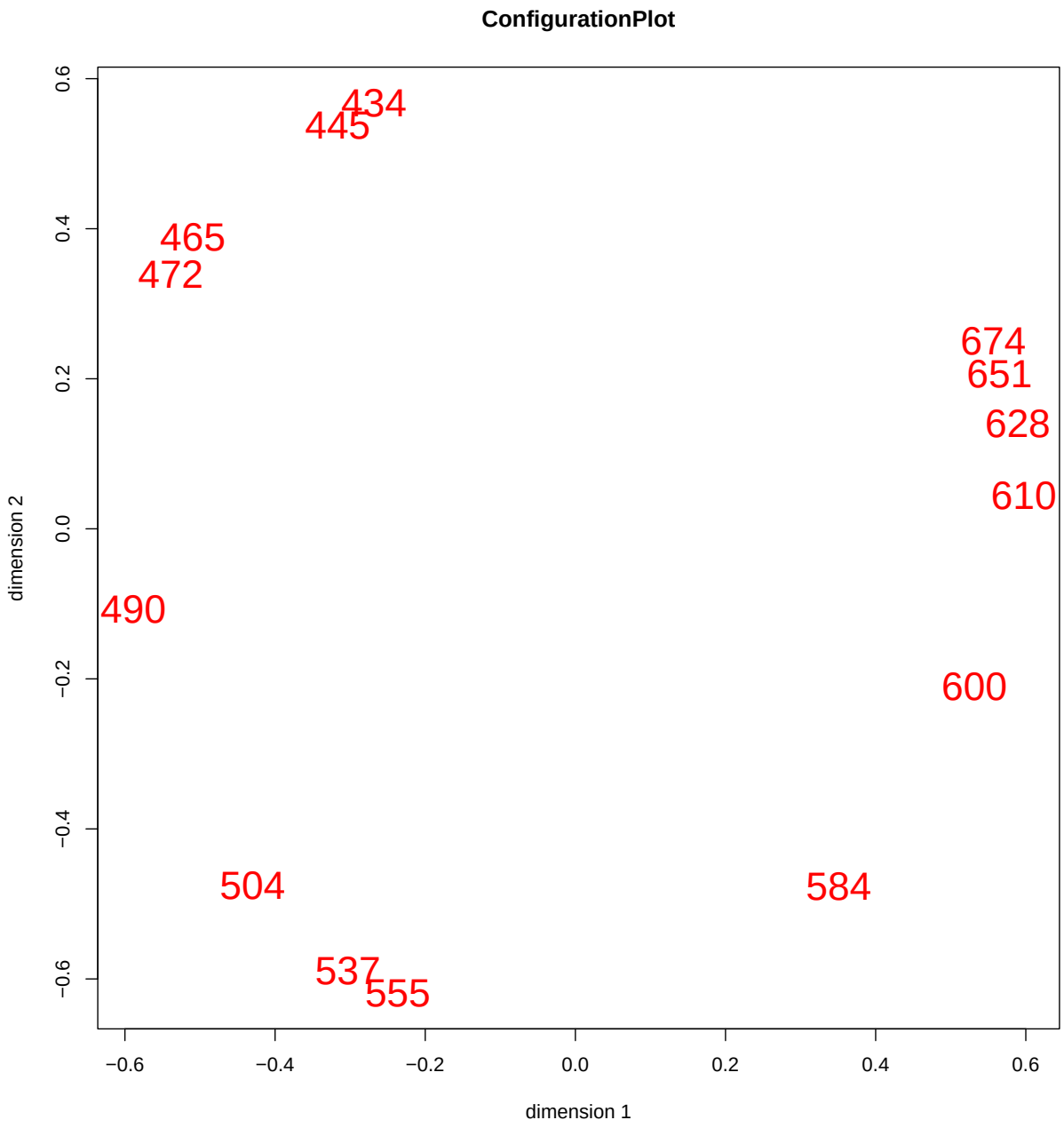

Figure 7: Configuration Plot, Metric, r = 2

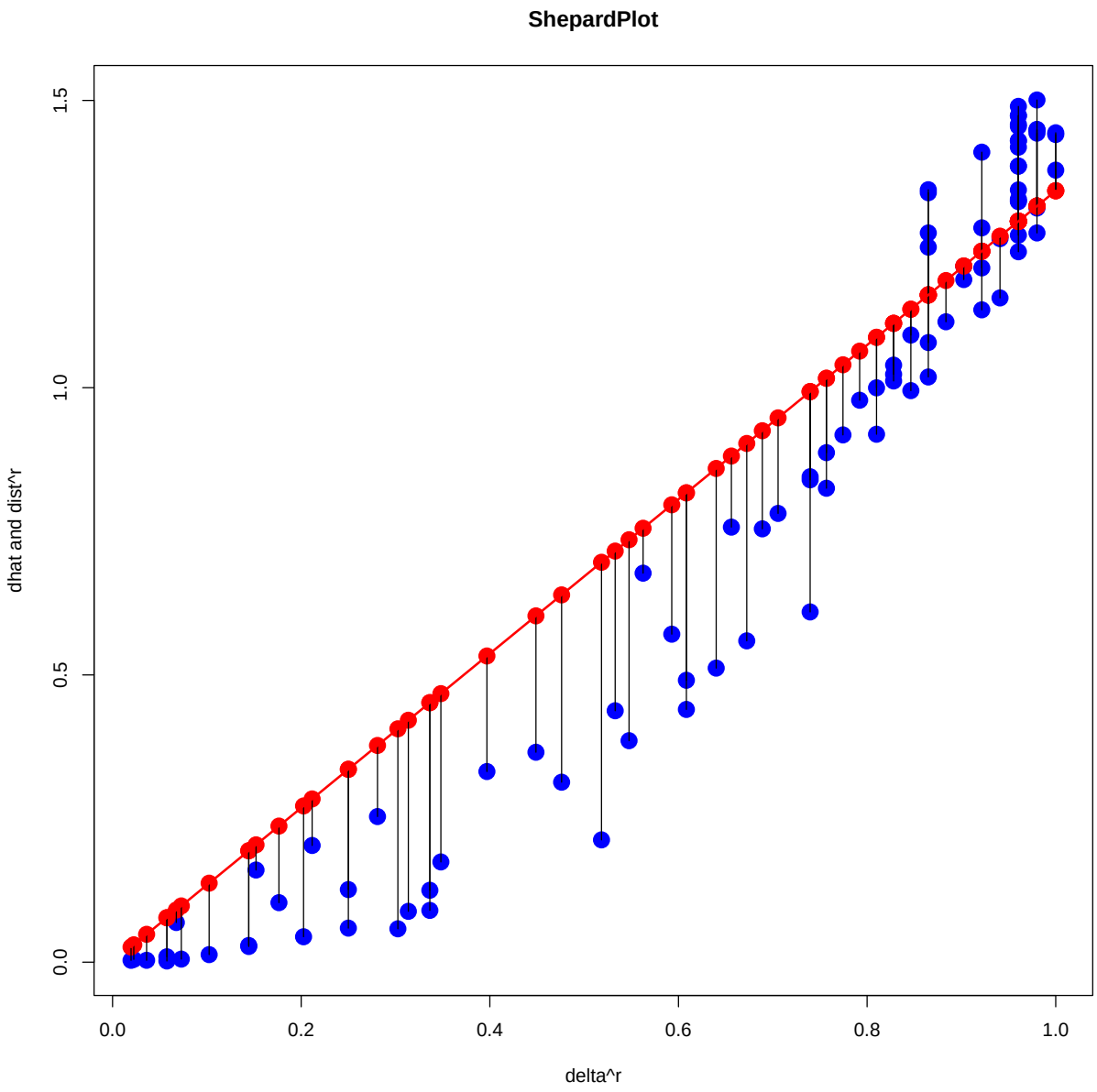

Figure 8: Shepard Plot, Metric, r = 2

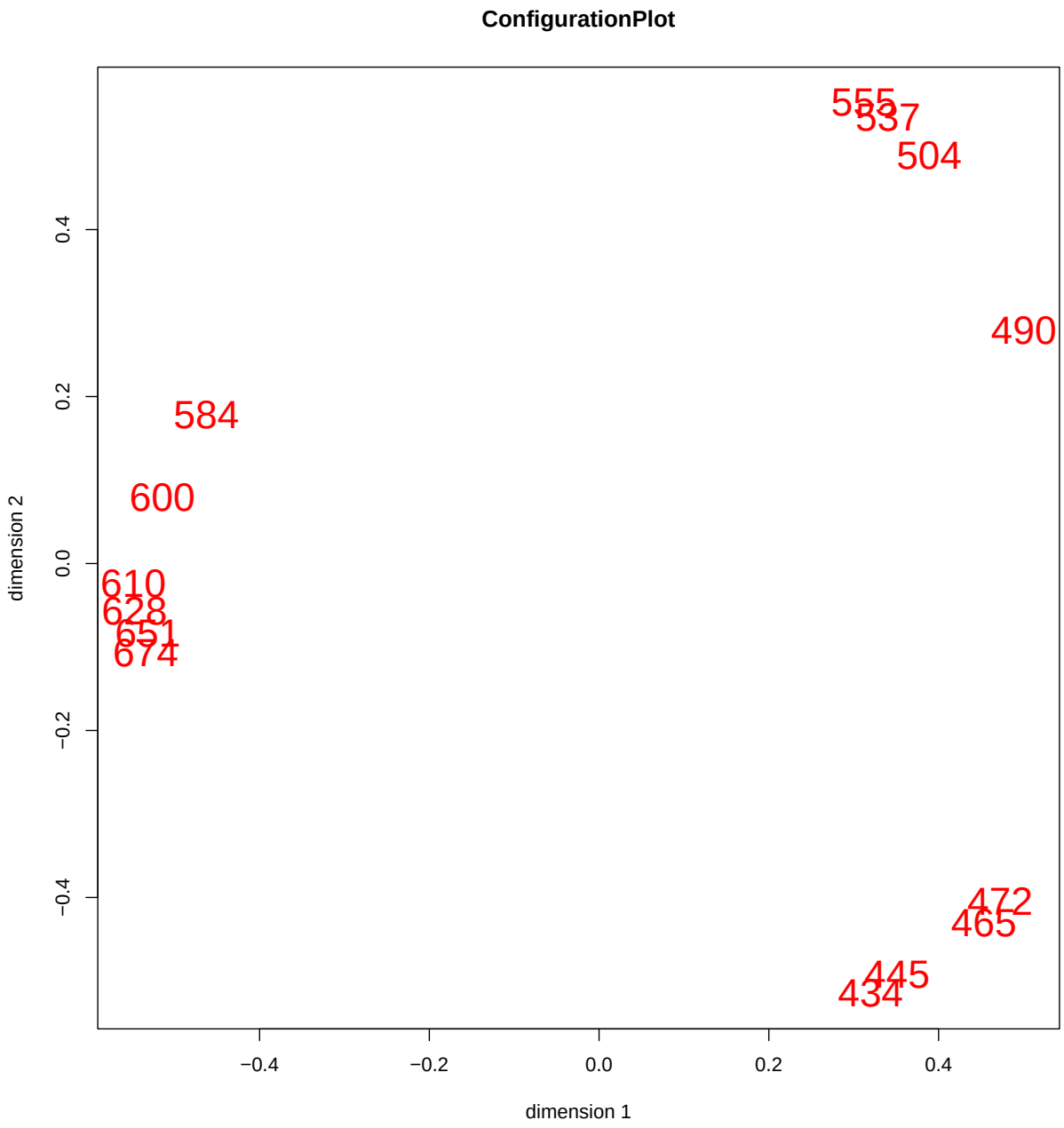

Figure 9: Configuration Plot, Metric, r = 5

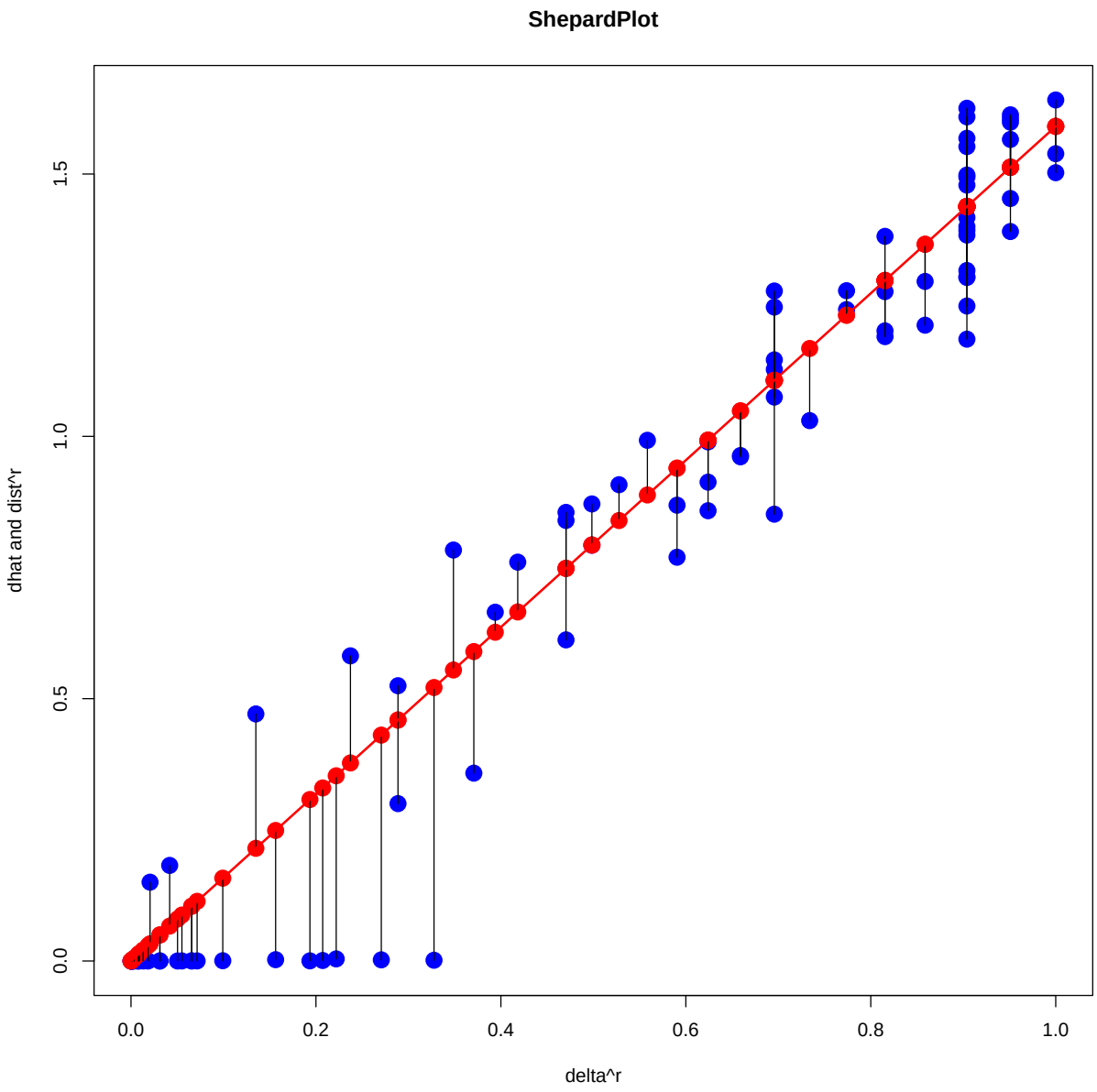

Figure 10: Shepard Plot, Metric, r = 5

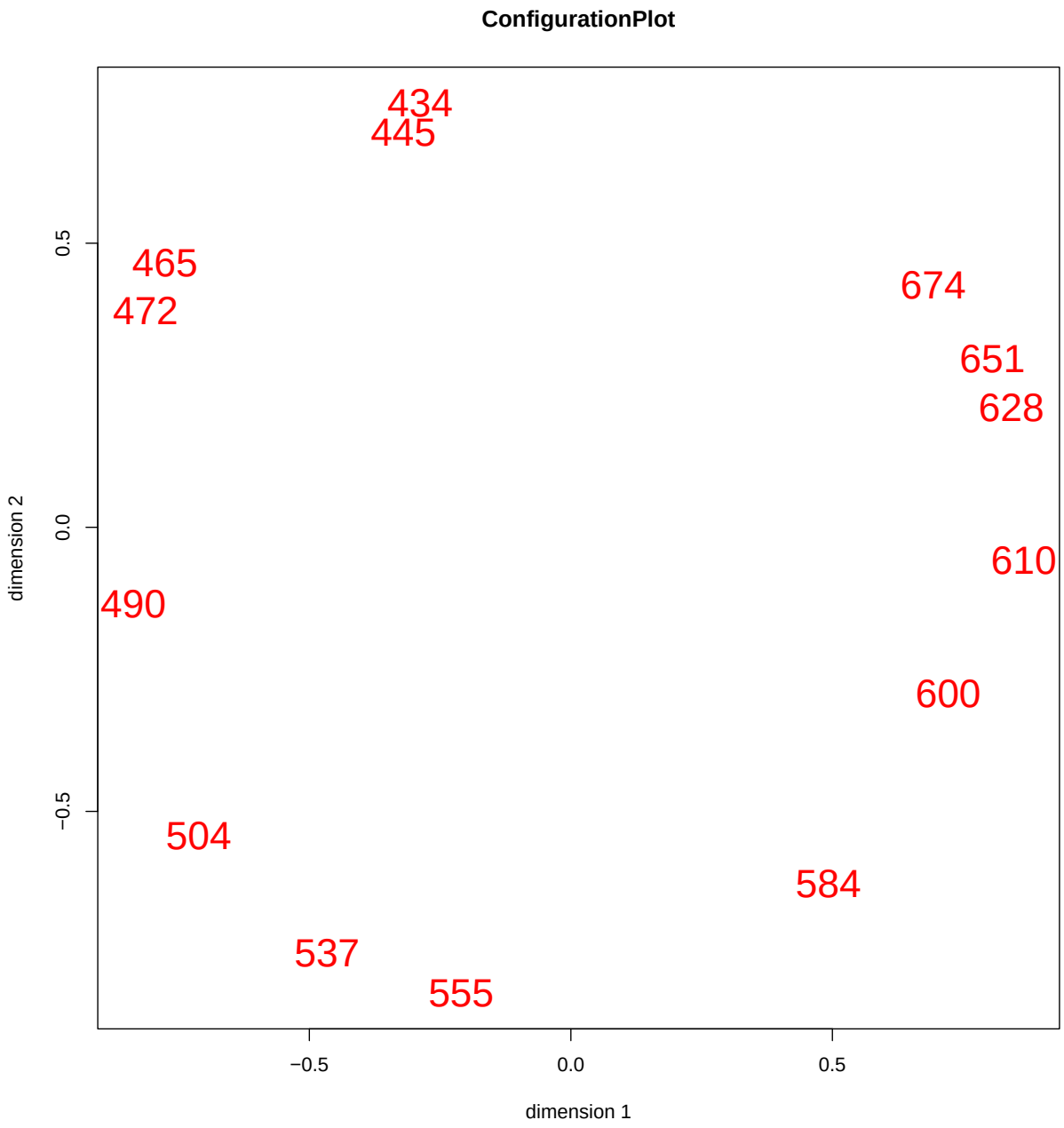

Figure 11: Configuration Plot, Non-metric, r = .1

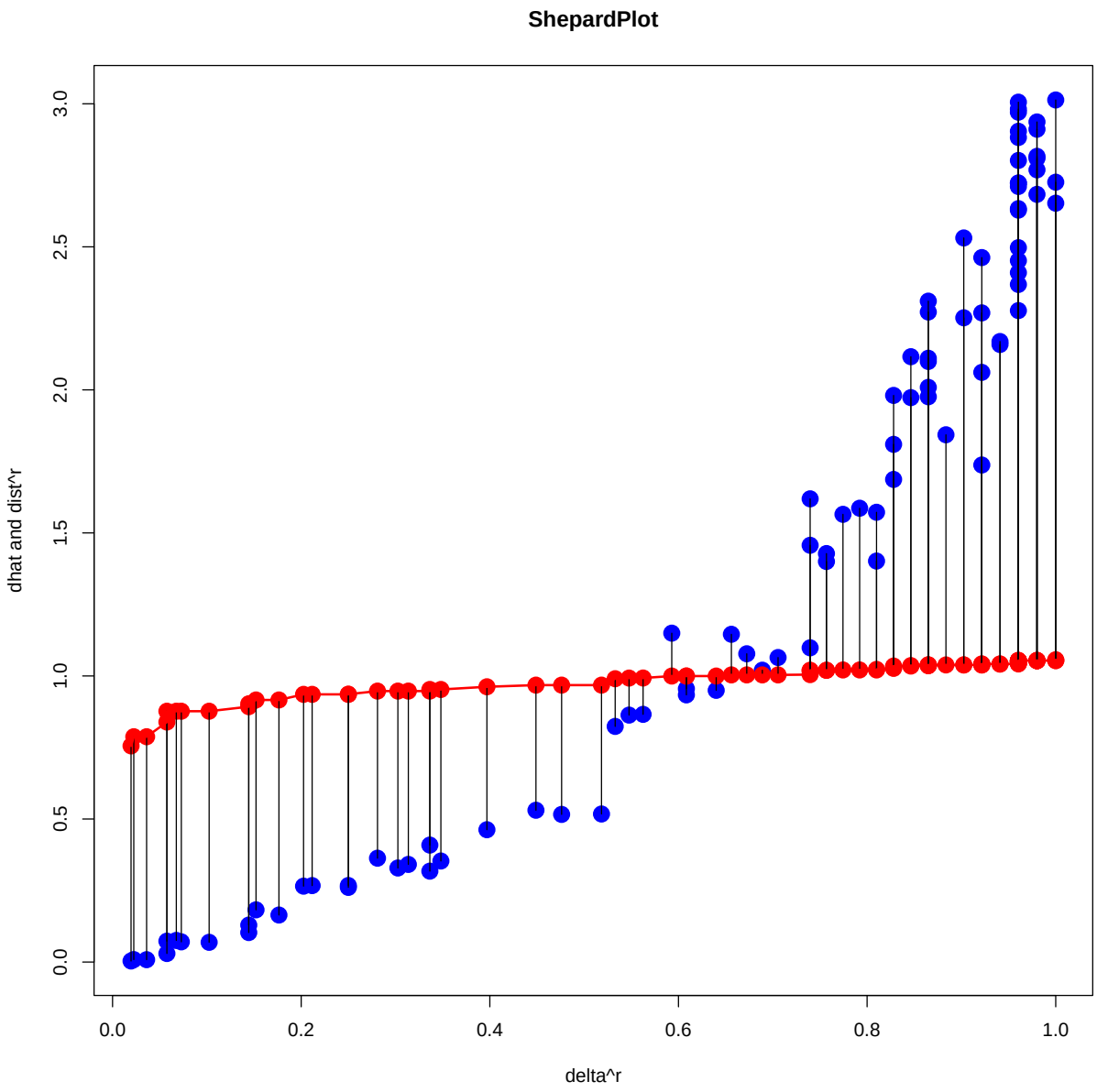

Figure 12: Shepard Plot, Non-metric, r = .1

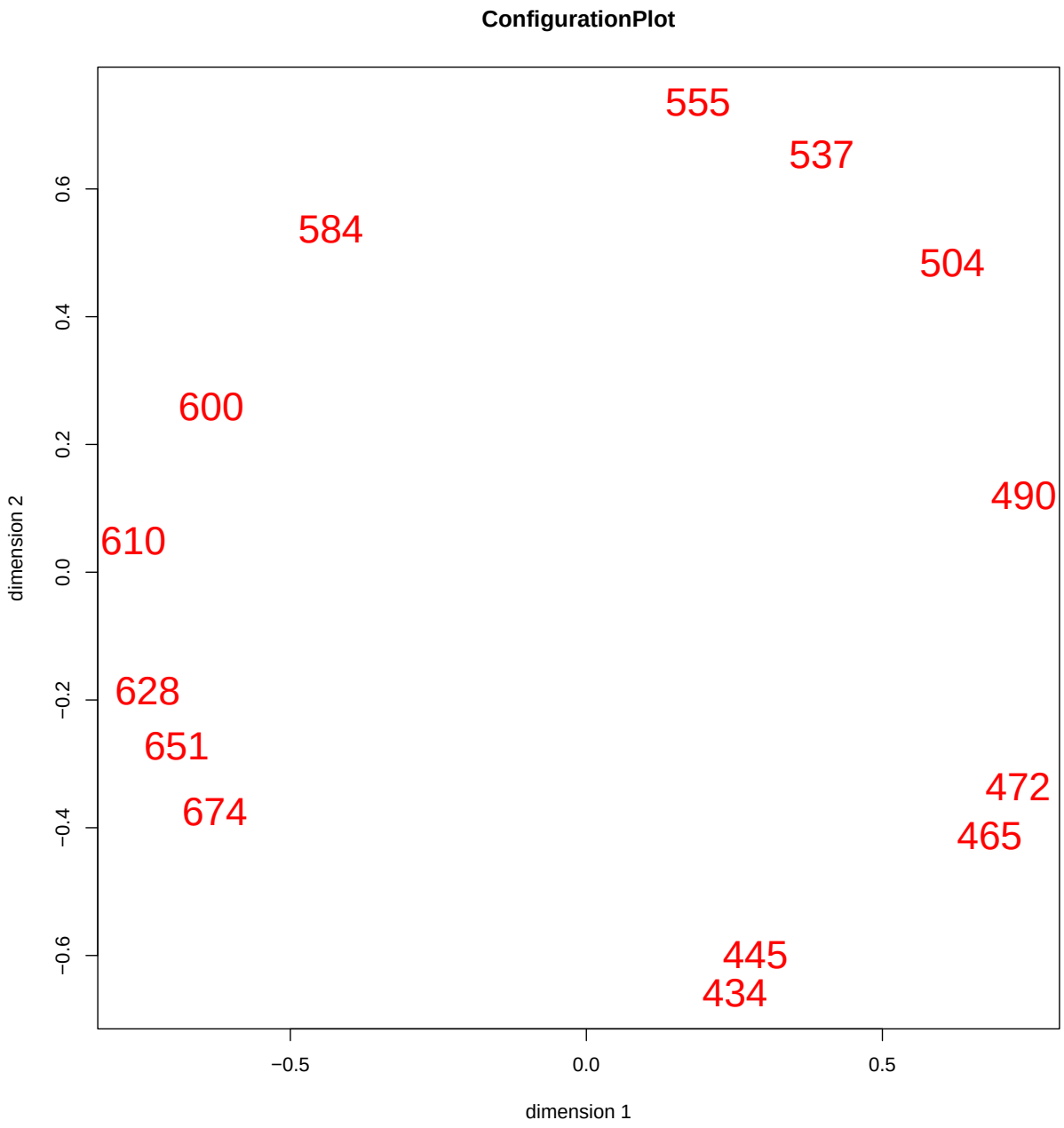

Figure 13: Configuration Plot, Non-metric, r = .5

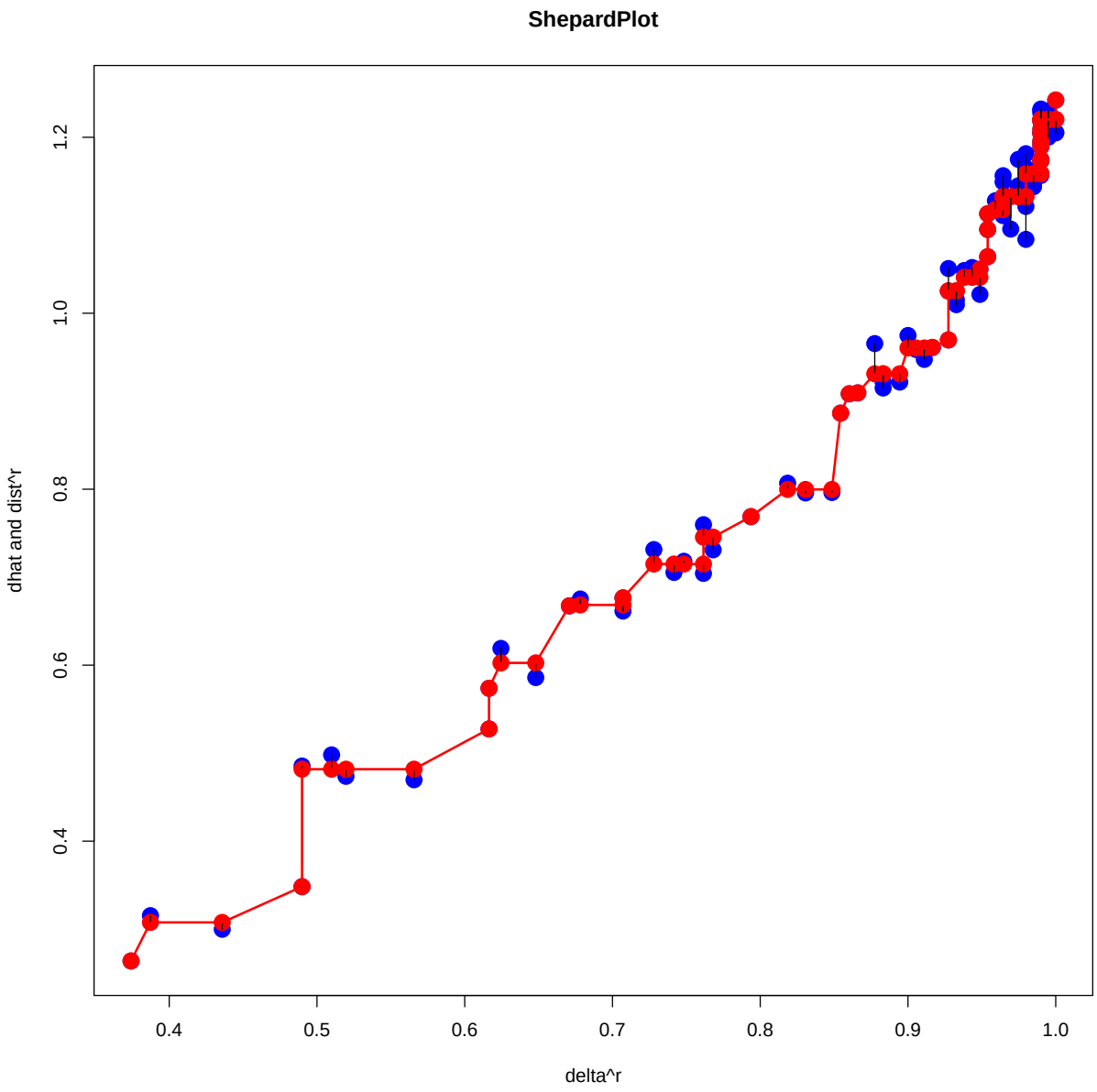

Figure 14: Shepard Plot, Non-metric, r = .5

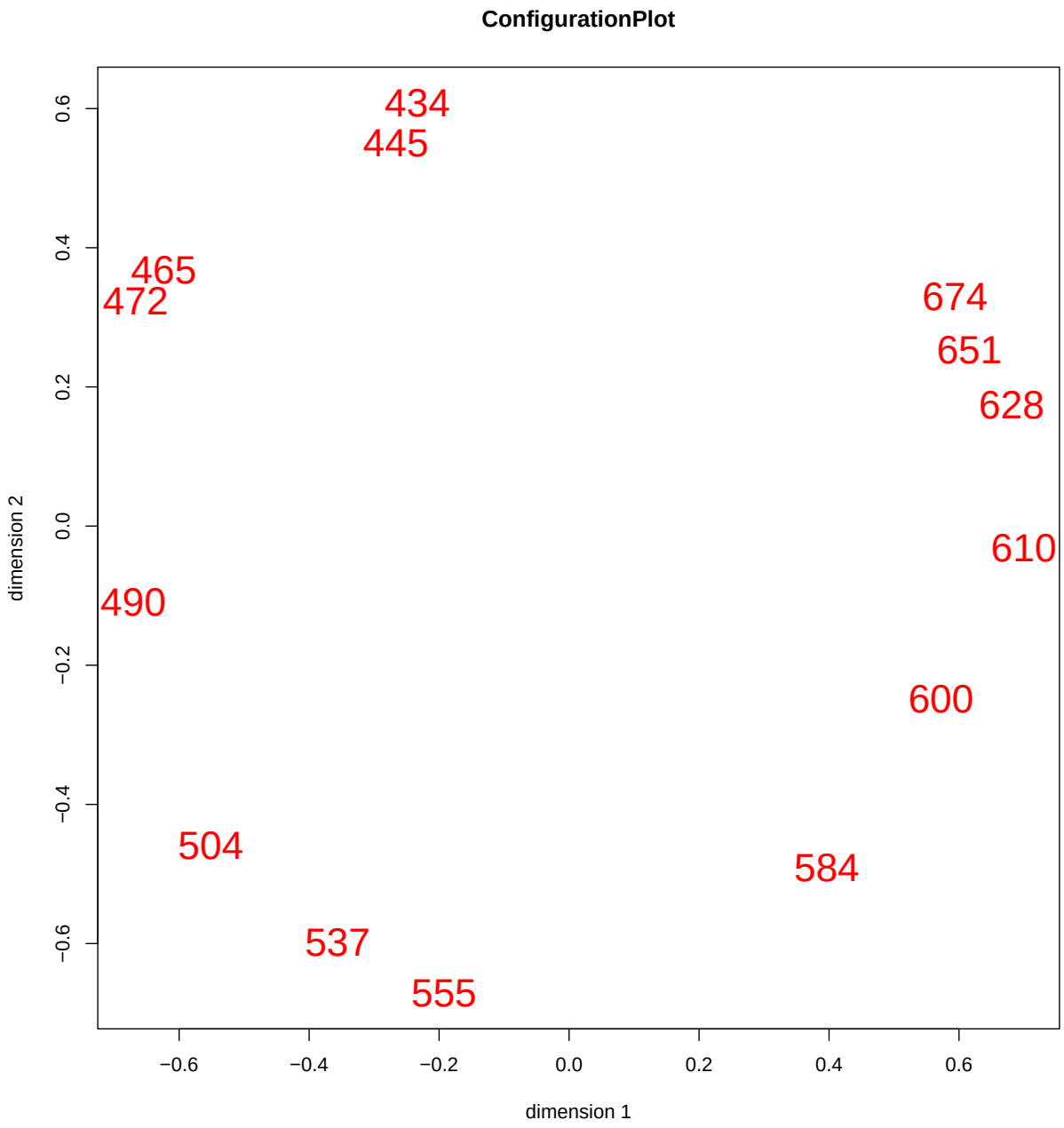

Figure 15: Configuration Plot, Non-metric, r = 1

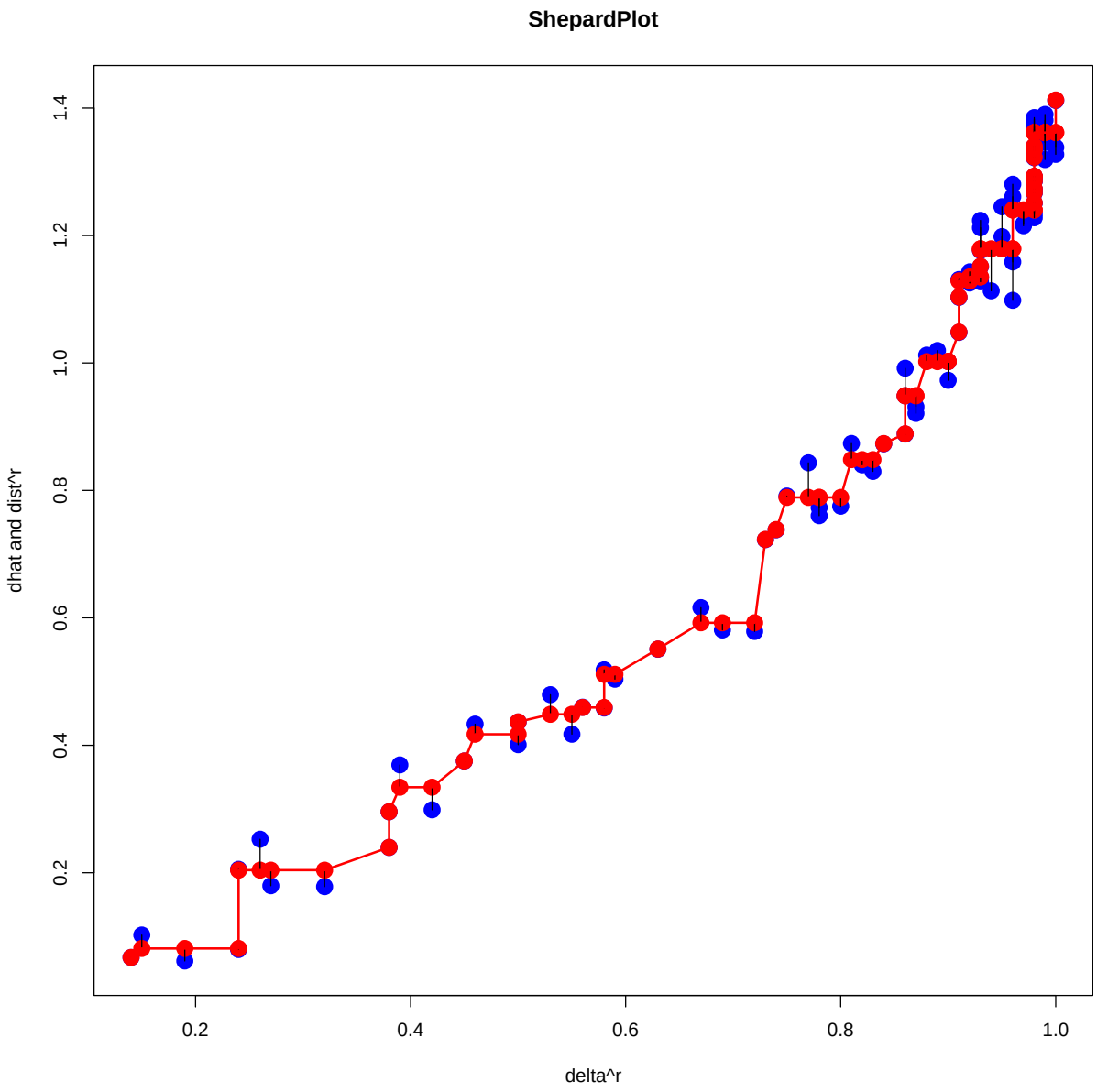

Figure 16: Shepard Plot, Non-metric, r = 1

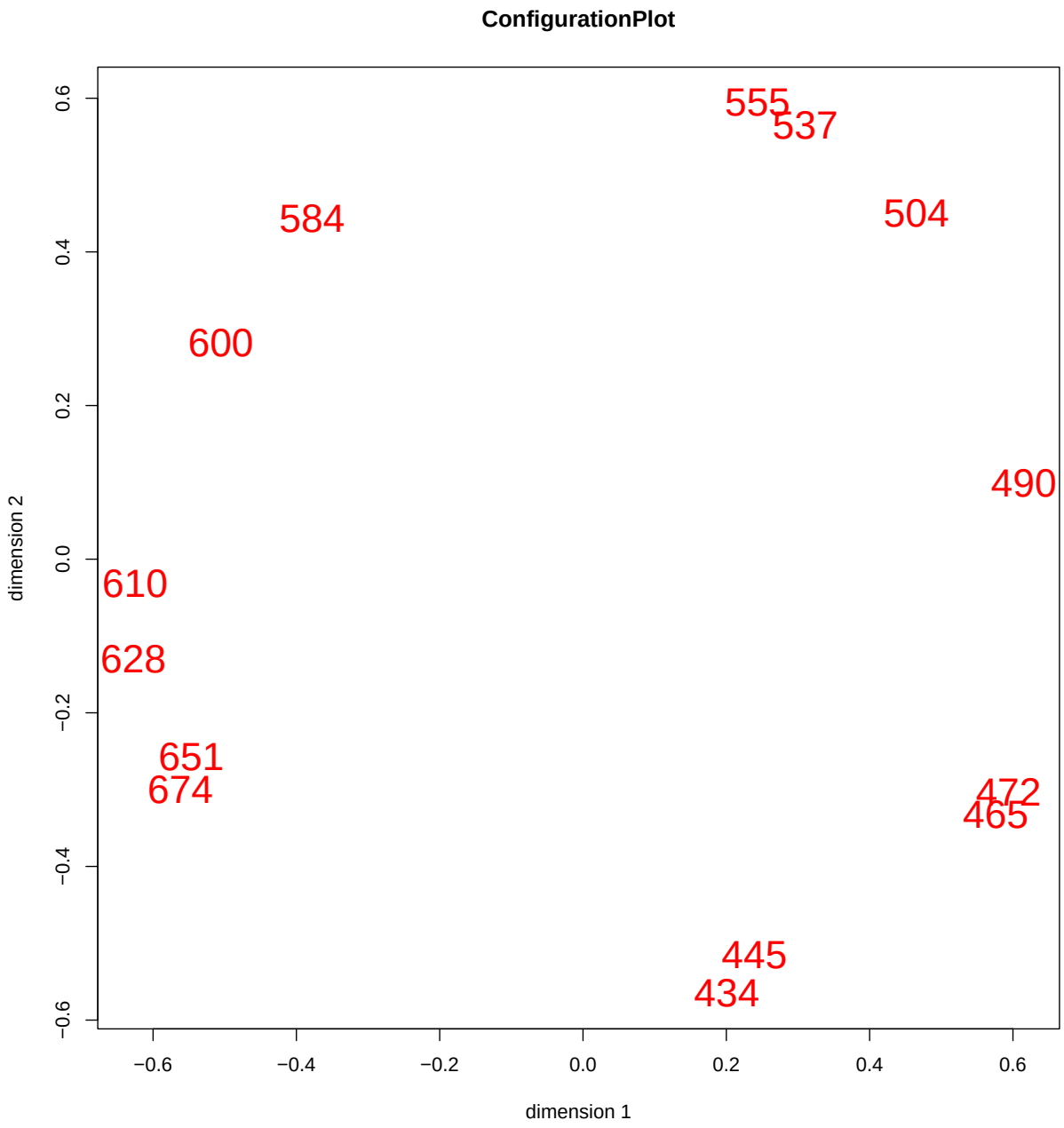

Figure 17: Configuration Plot, Non-metric, r = 2

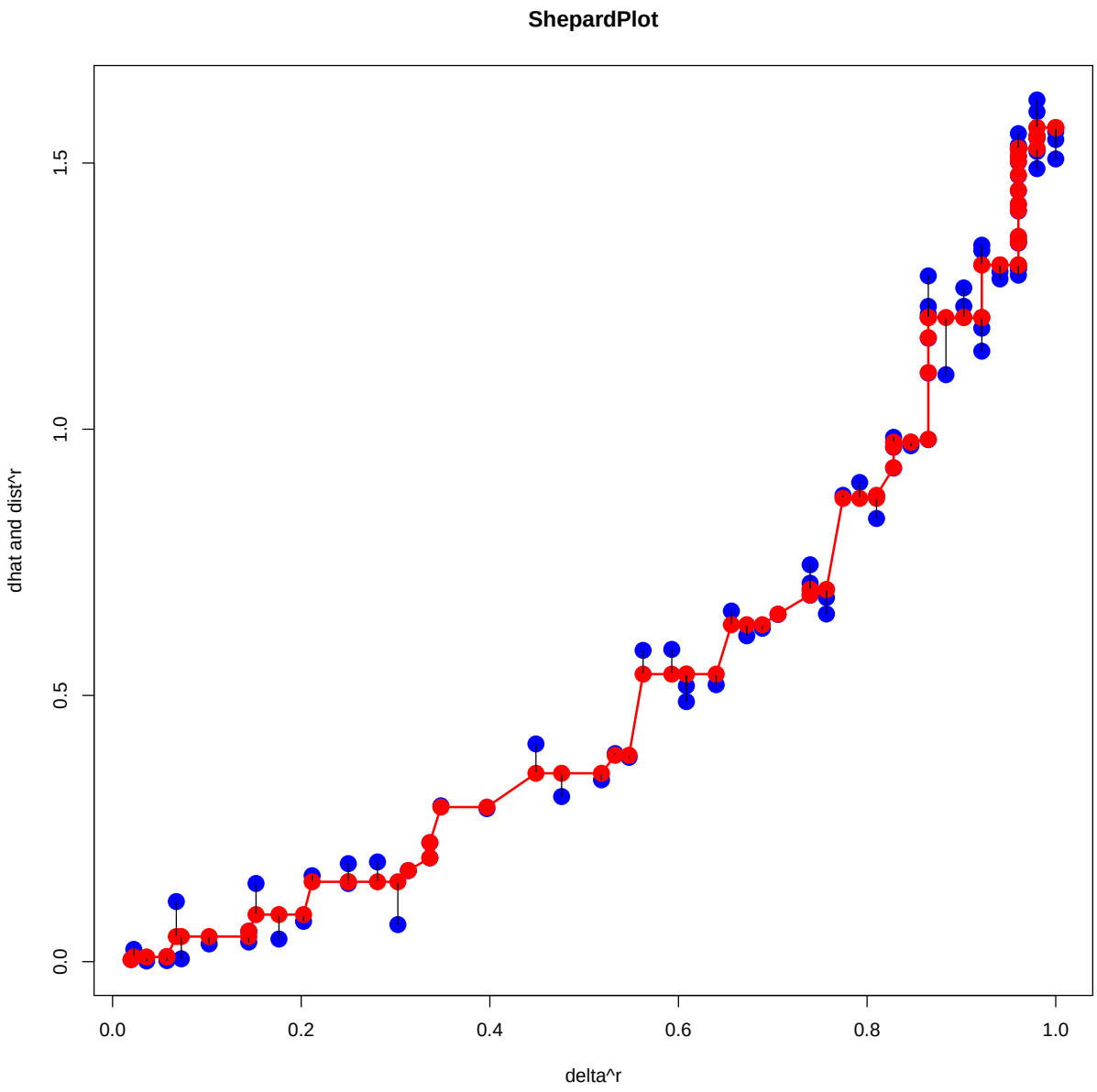

Figure 18: Shepard Plot, Non-metric, r = 2

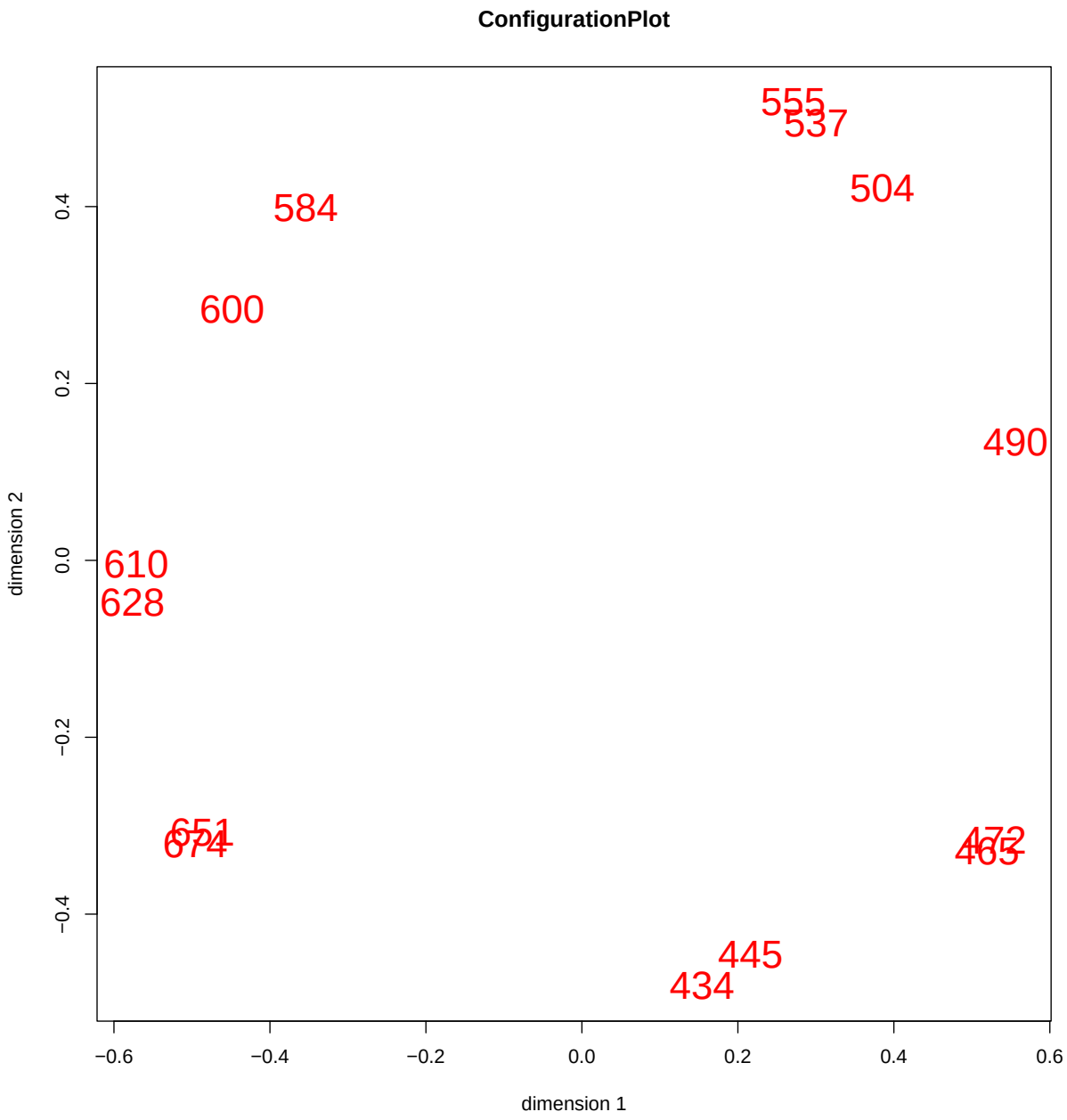

Figure 19: Configuration Plot, Non-metric, r = 5

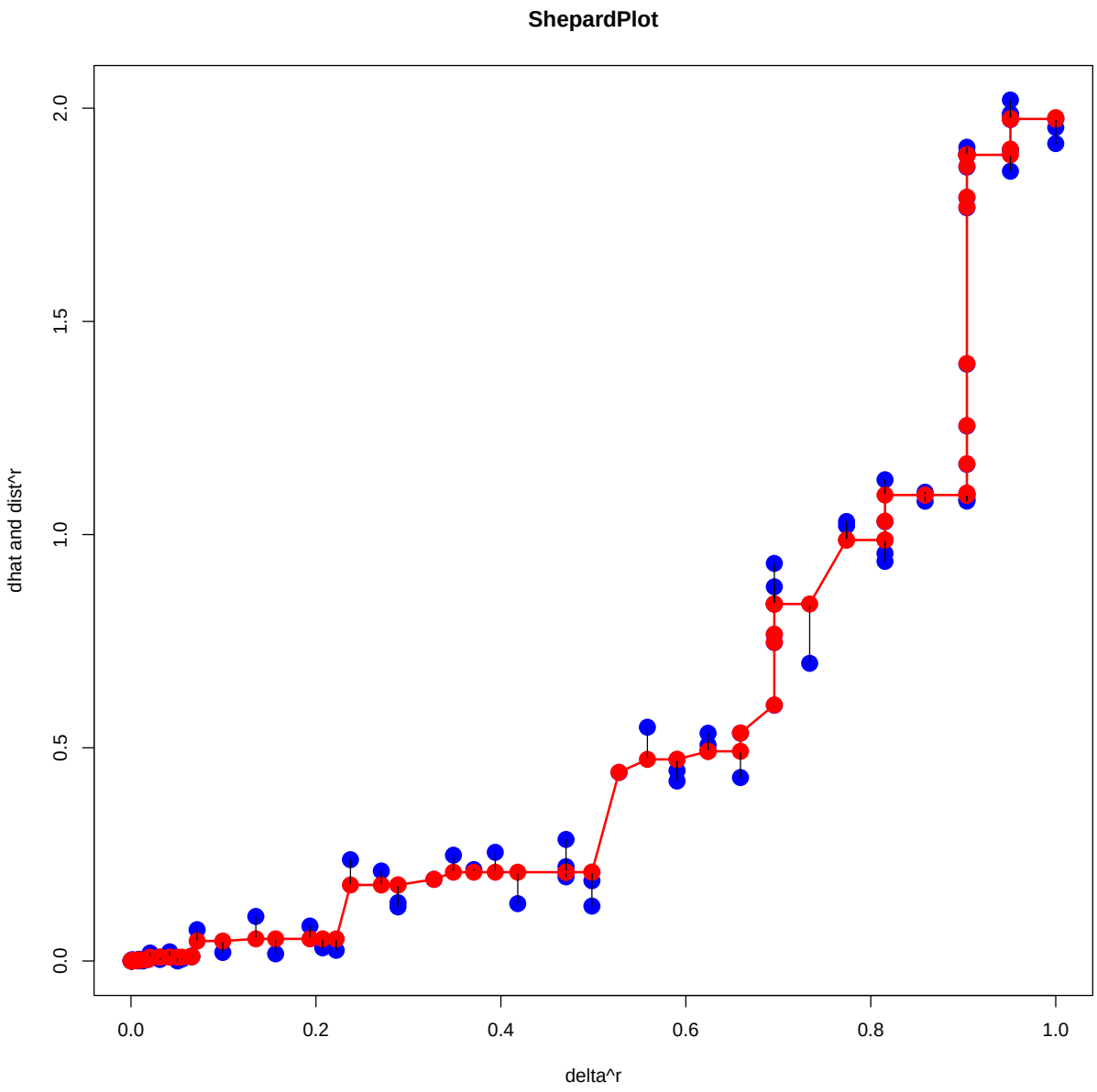

Figure 20: Shepard Plot, Non-metric, r = 5

References

Bauschke, Heinz H., Minh N. Bui, and Xianfu Wang. 2018. "Projecting onto the Intersection of a Cone and a Sphere." *SIAM Journal on Optimization* 28: 2158–88.

De Leeuw, Jan. 1975. "A Normalized Cone Regression Approach to Alternating Least Squares Algorithms." Department of Data Theory FSW/RUL. https://jansweb.netlify.app/publication/deleeuw-u-75-a/deleeuw-u-75-a.pdf.

———. 1977. "Applications of Convex Analysis to Multidimensional Scaling." In *Recent Developments in Statistics*, edited by J. R. Barra, F. Brodeau, G. Romier, and B. Van Cutsem, 133–45. Amsterdam, The Netherlands: North Holland Publishing Company.

———. 2017. "Higher Partials of fStress. Who Needs Them ?" 2017. https://jansweb.netlify.app/publication/deleeuw-e-17-r/deleeuw-e-17-r.pdf.

———. 2025. "Yet Another Smacof - Square Symmetric Case." 2025. https://jansweb.netlify.app/publication/deleeuw-e-25-b/.

De Leeuw, Jan, Patrick Groenen, and Patrick Mair. 2016. "Minimizing rStress Using Majorization." 2016. https://jansweb.netlify.app/publication/deleeuw-groenen-mair-e-16-a/deleeuw-groenen-mair-e-16-a.pdf.

De Leeuw, Jan, and Patrick Mair. 2009. "Multidimensional Scaling Using Majorization: SMACOF in R." *Journal of Statistical Software* 31 (3): 1–30. https://www.jstatsoft.org/article/view/v031i03.

Ekman, Gosta. 1954. "Dimensions of Color Vision." *Journal of Psychology* 38: 467–74.

Groenen, Patrick J. F., Jan De Leeuw, and Rudolf Mathar. 1995. "Least Squares Multidimensional Scaling with Transformed Distances." In *From Data to Knowledge: Theoretical and Practical Aspects of Classification, Data Analysis and Knowledge Organization*, edited by W. Gaul and D. Pfeifer. Berlin, Germany: Springer Verlag. https://jansweb.netlify.app/publication/groenen-deleeuw-mathar-c-95/groenen-deleeuw-mathar-c-95.pdf.

Heiser, Willem J. 1991. "A Generalized Majorization Method for Least Squares Multidimensional Scaling of Pseudodistances that May Be Negative." *Psychometrika* 56 (1): 7–27.

Kruskal, Joseph B. 1964a. "Multidimensional Scaling by Optimizing Goodness of Fit to a Nonmetric Hypothesis." *Psychometrika* 29: 1–27.

———. 1964b. "Nonmetric Multidimensional Scaling: a Numerical Method." *Psychometrika* 29: 115–29.

Mair, Patrick, Patrick J. F. Groenen, and Jan De Leeuw. 2022. "More on Multidimensional Scaling in R: smacof Version 2." *Journal of Statistical Software* 102 (10): 1–47. https://www.jstatsoft.org/article/view/v102i10.

Mersmann, Olaf. 2024. *microbenchmark: Accurate Timing Functions*. https://CRAN.R-project.org/package=microbenchmark.

Qiu, Yixuan, and Jiali Mei. 2024. *RSpectra: Solvers for Large-Scale Eigenvalue and SVD Problems*. https://CRAN.R-project.org/package=RSpectra.

Ramsay, James O. 1977. "Maximum Likelihood Estimation in Multidimensional Scaling." *Psychometrika* 42: 241–66.

Rusch, Thomas, Jan De Leeuw, Patrick Mair, and Kurt Hornik. In Press. "Flexible Multidimensional Scaling with the r Package Smacofx." *Journal of Statistical Software*, In Press. https://jansweb.netlify.app/publication/rusch-deleeuw-mair-hornik-a-25/rusch-deleeuw-mair-hornik-a-25.pdf.

Takane, Yoshio, Forrest W. Young, and Jan De Leeuw. 1977. "Nonmetric Individual Differences in Multidimensional Scaling: An Alternating Least Squares Method with Optimal Scaling Features." *Psychometrika* 42: 7–67.